DATABASE DESIGN LAB ASSIGNMENT

SQL JOIN Operations

Course: CSE 364.7 — Database Design Lab
Faculty: MD. NASIM ADNAN

Prepared by

Abid Hasan

ID:2023200000720

Batch: 65

Section : 07

Date: 10 June 2026

Southeast University

Department of Computer Science and Engineering

1. Conceptual Foundations — What is a JOIN?

In a normalized relational database, related information is deliberately spread across multiple tables to avoid redundancy. A JOIN is the SQL operation that re-combines these tables for querying by matching rows based on a related column (typically a foreign key relationship). Understanding the type of join you need is essential: the wrong join type either loses data silently or introduces unexpected NULL rows into your results.

The database and schema utilized throughout this assignment are the same as those practiced and established during the lab sessions. All queries execute against this established schema.

Primary Tables and Schema Definitions

The database schema is designed to manage employee, academic staff, managerial, and organizational records. The following tables are included:

Employee

Stores core employee information and salary details.

- **Ids :** INT (Primary Key)
- **Name :** VARCHAR(50)
- **Gender :** CHAR(1)
- **Date_of_Birth :** DATE
- **Date_of_Join :** DATE
- **Permanent_Address :** VARCHAR(255)
- **Current_Address :** VARCHAR(255)
- **Phone :** VARCHAR(15)
- **Type :** CHAR(1)
- **Basic_Salary :** DECIMAL(7,2)
- **Others :** DECIMAL(7,2)
- **Dept_Id :** INT (Foreign Key)
- **Desig_Id :** INT (Foreign Key)

Department

Stores information about organizational departments.

- **Dept_Id :** INT (Primary Key)
- **Name :** VARCHAR(10)

Designation

Stores employee designation or job rank information.

- **Desig_Id :** INT (Primary Key)
- **Name :** VARCHAR(20)

Manager

Stores information about employees with managerial responsibilities.

- **Ids :** INT (Primary Key)
- **Name :** VARCHAR(10)
- **Gender :** CHAR(1)
- **Permanent_Address :** VARCHAR(255)
- **Current_Address :** VARCHAR(255)
- **Phone :** VARCHAR(50)
- **Type :** CHAR(1)
- **Managerial_Allowance :** DECIMAL(7,2)
- **Dept_Id :** INT (Foreign Key)

Teacher_Details

Stores additional information related to teaching staff.

- **Ids :** INT (Primary Key)
- **E_Mail:** VARCHAR(25)
- **Papers :** INT

Officer_Details

Stores additional information related to administrative officers.

- **Ids :** INT (Primary Key)
- **Desk_No :** INT
- **Association_Member :** CHAR(1)

Primary Key and Foreign Key Relationships

The following integrity constraints maintain referential consistency across the database:

1. **Departmental Relationship**
   Employee.Dept_Id (FK) references Department.Dept_Id (PK).
2. **Designation Relationship**
   Employee.Desig_Id (FK) references Designation.Desig_Id (PK).
3. **Managerial Relationship**
   Manager.Dept_Id (FK) references Department.Dept_Id (PK).
4. **Teacher–Employee Relationship**
   Teacher_Details.Ids (FK) references Employee.Ids (PK).

5. **Officer–Employee Relationship**
   Officer_Details.Ids (FK) references Employee.Ids (PK).
6. **Teacher–Department Relationship**
   Teacher_Details is associated with the Department to identify the department of academic staff.
7. **Officer–Designation Relationship**
   Officer_Details is associated with Designation to identify the designation of administrative officers.

This schema ensures data consistency while supporting employee management, departmental organization, academic staff records, and administrative officer information.

1.1 Types of SQL Joins

Join Type	Returns	Rows Included
INNER JOIN	Matched rows only	Only rows with a match in BOTH tables
NATURAL JOIN	Matched on shared column names	Same as INNER JOIN but column matching is automatic
LEFT JOIN	All left + matched right	All rows from left table; NULL if no right match
RIGHT JOIN	All right + matched left	All rows from right table; NULL if no left match
FULL JOIN	All rows from both tables	All rows from both; NULLs wherever no match exists

Important Note : MySQL does NOT support FULL OUTER JOIN or MINUS or INTERSECT set operations natively. The workaround is UNION of a LEFT JOIN and a RIGHT JOIN.

2: INNER JOIN

INNER JOIN : Inner join is a join that retrieves only the matching rows from both tables based on a specified condition. It returns records where the join condition is satisfied in both tables; non-matching rows are excluded from the result**. NATURAL JOIN is a special form of INNER JOIN.**

2.1 How INNER JOIN Works

When you perform an `INNER JOIN`, the database engine compares each row from the first table against every row of the second table and returns only those pairs where the join condition evaluates to true. In our schema, `Employee` and `Department` both contain the column

`Dept_Id`, so joining on that column returns only employees who belong to an existing department.

There are two syntactically different but logically equivalent ways to write an `INNER JOIN`:

- **Implicit style** — tables listed in `FROM`, condition in `WHERE`
- **Explicit style** — using the `INNER JOIN ... ON` keyword

Query 1 : Basic INNER JOIN — Employee with their Department Name

Goal: Retrieve each employee's name alongside their department name.

Method 1 — Implicit Join (WHERE clause):

```
--Query
SELECT
   E.Ids,
   E.Name  AS Employee_Name,
   E.Type  AS Employee_Type,
   E.Basic_salary,
   D.Name  AS Department_Name
FROM Employee E, Department D
WHERE E.Dept_Id = D.Dept_Id;
```

Method 2 — Explicit INNER JOIN:

```
--Query
SELECT
   E.Ids,
   E.Name  AS Employee_Name,
   E.Type  AS Employee_Type,
   E.Basic_salary,
   D.Name  AS Department_Name
FROM Employee E
INNER JOIN Department D ON E.Dept_Id = D.Dept_Id;
```

Output Table :

Ids	Employee_Name	Employee_Type	Basic_salary	Department_Name
1	Nasim	T	15000	CSE
2	Arshad	T	12000	BBA
3	Fahima	T	12000	ETE
4	Nowrin	O	10000	CSE
5	Rafiq	T	45000	CSE
6	Salam	T	20000	BBA
7	Zia	T	15000	ETE

WHY THIS MATTERS: In a real database, employee and department data are stored in separate tables to avoid redundancy. Without a join, you cannot see which department an employee belongs to in a single query. `INNER JOIN` bridges these tables and returns only valid, matched records — employees without a department and departments without any employee are both excluded, ensuring clean and meaningful output.

Query 2 — INNER JOIN with Additional Filtering

Goal: Show only teachers (`Type = 'T'`) with their department, sorted by gross salary descending.

Method 1 — Implicit Join (WHERE clause):

```
--sql
SELECT
   E.Name  AS Teacher_Name,
   E.Basic_salary,
   E.Others,
   (E.Basic_salary + E.Others)   AS Total_Salary,
   D.Name AS Department
   FROM Employee E, Department D
WHERE E.Dept_Id = D.Dept_Id
AND E.Type = 'T'
ORDER BY Total_Salary DESC;
```

Method 2 — Explicit INNER JOIN:

```sql
--sql
SELECT
   E.Name AS Teacher_Name,
   E.Basic_salary,
   E.Others,
   (E.Basic_salary + E.Others)   AS Total_Salary,
   D.Name  AS Department
FROM Employee E
INNER JOIN Department D ON E.Dept_Id = D.Dept_Id
WHERE E.Type = 'T'
ORDER BY Total_Salary DESC;
```

Output Table :

Teacher_Name	Basic_salary	Others	Total_Salary	Department
Rafiq	45000	15000	60000	CSE
Salam	20000	5000	25000	BBA
Nasim	15000	5000	20000	CSE
Zia	15000	1700	16700	ETE
Fahima	12000	3500	15500	ETE
Arshad	12000	3000	15000	BBA

WHY THIS MATTERS: Combining `INNER JOIN` with `WHERE` and `ORDER BY` demonstrates the real power of SQL you can filter, join, and sort in a single query. Here, computing `Total_Salary` as `(Basic_salary + Others)` directly inside the query eliminates the need for any post-processing, and sorting by it immediately reveals the highest-paid teachers useful for payroll analysis or faculty reports.

Query 3 — Three-Table INNER JOIN — Employee, Department, and Designation

Goal: Display each employee's name, department, and designation together in a single query.

Method 1 — Implicit Join (WHERE clause):

```sql
--sql
SELECT
   E.Name  AS Employee_Name,
   D.Name  AS Department,
   DG.Name  AS Designation,
   E.Basic_salary  AS Basic_Salary
FROM Employee E, Department D, Designation DG
WHERE E.Dept_Id  = D.Dept_Id
AND E.Desig_Id = DG.Desig_Id
ORDER BY D.Name, E.Basic_salary DESC;
```

Method 2 — Explicit INNER JOIN:

```sql
--sql
SELECT
   E.Name AS Employee_Name,
   D.Name AS Department,
   DG.Name AS Designation,
   E.Basic_salary  AS Basic_Salary
FROM Employee E
INNER JOIN Department D   ON E.Dept_Id  = D.Dept_Id
INNER JOIN Designation DG ON E.Desig_Id = DG.Desig_Id
ORDER BY D.Name, E.Basic_salary DESC;
```

Output Table:

Employee_Name	Department	Designation	Basic_Salary
Salam	BBA	Associate Professor	20000
Arshad	BBA	Lecturer	12000
Rafiq	CSE	Professor	45000
Nasim	CSE	Assistant Professor	15000
Nowrin	CSE	Officer	10000
Zia	ETE	Assistant Professor	15000
Fahima	ETE	Associate Professor	12000

Chaining multiple `INNER JOIN`s allows querying across three or more tables in a single statement. The implicit style handles this by listing all tables in `FROM` and combining all join conditions in the `WHERE` clause with `AND`.

WHY THIS MATTERS: In a normalized database, related information is deliberately split across multiple tables. A three-table `INNER JOIN` reconstructs a complete, human-readable employee profile — name, department, and designation from those separate pieces in one query. This is the foundation of how real-world HR systems, reporting dashboards, and admin panels fetch and display structured data efficiently.

Note:

Natural JOIN:

It is worth nothing that `NATURAL JOIN` is simply a special form of `INNER JOIN`. While a standard `INNER JOIN` requires the user to explicitly define the join condition, `NATURAL JOIN` removes that requirement by automatically detecting columns that share the same name across both tables. The result is identical to writing an `INNER JOIN` with those columns manually specified. In other words, every `NATURAL JOIN` is an `INNER JOIN` under the hood, but not every `INNER JOIN` can be expressed as a `NATURAL JOIN` only when the joining columns happen to share the same name in both tables.

For example, the following two queries are intended to produce the same result:

INNER JOIN (explicit):

```sql
--sql
SELECT
   E.Name  AS Employee_Name,
   D.Name  AS Department_Name
FROM Employee E
INNER JOIN Department D ON E.Dept_Id = D.Dept_Id;
```

Output table:

Employee_Name	Department_Name
Nasim	CSE
Nowrin	CSE
Rafiq	CSE
Arshad	BBA
Salam	BBA

Fahima	ETE
Zia	ETE

NATURAL JOIN (auto-detected):

```
--sql
SELECT
   E.Name AS Employee_Name,
   D.Name  AS Department_Name
FROM Employee E
NATURAL JOIN Department D;
```

Output table:

Empty table

However, in our schema both `Employee` and `Department` contain a column named `Name`. Because of this, `NATURAL JOIN` silently constructs the join condition as `E.Dept_Id = D.Dept_Id AND E.Name = D.Name` matching on both columns simultaneously. This causes most rows to be excluded from the result, producing incorrect and incomplete output. The explicit `INNER JOIN` on the other hand joins strictly on `Dept_Id` as intended, returning all correct rows.

THIS IS WHY `NATURAL JOIN` IS DANGEROUS: It is highly sensitive to column naming across tables. Any coincidental name match such as both tables having a `Name` column will silently add an unintended join condition, breaking the query without any error or warning. For this reason, explicit `INNER JOIN` with an `ON` clause is always the safer and more reliable choice.

3. LEFT JOIN (LEFT OUTER JOIN)

LEFT JOIN: Returns ALL rows from the LEFT table, plus matching rows from the RIGHT table. Where no match exists in the right table, the right-side columns are filled with NULL.

3.1 When LEFT JOIN is Essential

LEFT JOIN is critical when you need a complete list from one table and want to annotate it with optionally available information from another. Classic use cases include:

- Listing all employees, even those not yet assigned to a subclass (e.g., not in Teacher_Details)
- Listing all departments, including those with zero employees
- Producing reports where absence of data (NULL) is itself meaningful

QUERY 4 — LEFT JOIN — All Employees, Including Those Without Teacher DetailsGoal: Show every employee, and if they are a teacher, show their email and paper count

```
--sql
-- LEFT JOIN: all Employee rows are preserved
-- Employees who are NOT teachers get NULL in E_mail and Papers
SELECT
   E.Name  AS Employee_Name,
   E.Type,
   E.Basic_salary,
   TD.E_mail  AS Teacher_Email,
   TD.Papers  AS Research_Papers
FROM Employee E
LEFT JOIN Teacher_Details TD ON E.Ids = TD.Ids
ORDER BY E.Type, E.Basic_salary DESC;
```

Expected: 7 rows returned (all employees)

Employees 4 (Nowrin, Officer) will show NULL in Teacher_Email and Research_Papers

because there is no matching record in Teacher_Details for Ids = 4

Output table:

Employee_Name	Type	Basic_salary	Teacher_Email	Research_Papers
Nowrin	O	10000	Null	Null
Rafiq	T	45000	rafiq@abc.ac.bd	20
Salam	T	20000	salam@abc.ac.bd	9
Nasim	T	15000	nasim_1547@yahoo.co.uk	5
Zia	T	15000	zia_arman@abc.ac.bd	2
Arshad	T	12000	arshad_parvez@yahoo.com	4
Fahima	T	12000	fahima_noor@abc.ac.bd	2

WHY THIS MATTERS: An INNER JOIN here would silently drop employee Nowrin (Ids=4) from the results, because she has no Teacher_Details record. LEFT JOIN reveals this absence explicitly through NULL.

QUERY 5 — LEFT JOIN — All Departments Including Empty Ones

Goal: Insert new department 'ICT' with id = 4 and show every department's average salary departments with no staff show 0, not NULL.

```
--sql
-- Department 'ICT' (Dept_Id=4) has no employees
-- LEFT JOIN preserves it; IFNULL converts NULL average to 0
insert into Department values (4,'ICT');

SELECT
   D.Name AS Department_Name,
   COUNT(E.Ids) AS Employee_Count,
   IFNULL(AVG(E.Basic_salary), 0) AS Avg_Basic_Salary,
   IFNULL(SUM(E.Basic_salary), 0) AS Total_Payroll
FROM Department D
LEFT JOIN Employee E ON D.Dept_Id = E.Dept_Id
GROUP BY D.Dept_Id, D.Name
ORDER BY Avg_Basic_Salary DESC;
```

Output Table:

Department_Name	Employee_Count	Avg_Basic_Salary	Total_Payroll
CSE	3	23333.33333	70000
BBA	2	16000	32000
ETE	2	13500	27000
ICT	0	0	0

WHY THIS MATTERS: Without LEFT JOIN, the ICT department would vanish from the report entirely giving a misleading picture of department existence and causing inconsistency in administrative dashboards.

QUERY 6 — LEFT JOIN — Finding Unmatched Records (Anti-Join Pattern)

Goal: Find all employees who are NOT registered as teachers

```
-- Powerful pattern: LEFT JOIN + WHERE right-side IS NULL
-- This isolates rows from the left table with NO matching right row
SELECT
   E.Ids,
   E.Name  AS Employee_Name,
   E.Type,
   E.Dept_Id
FROM Employee E
```

```
LEFT JOIN Teacher_Details TD ON E.Ids = TD.Ids
WHERE TD.Ids IS NULL;   -- Only employees with no Teacher_Details record
```

Expected: Returns Nowrin (Ids=4, Type='O') and any other non-teachers

TD.Ids IS NULL means the LEFT JOIN found no match this employee is NOT a teacher

WHY THIS MATTERS: The LEFT JOIN + IS NULL pattern is called an Anti-Join. It is the standard SQL technique for finding records in one table that have no corresponding record in another equivalent to a conceptual MINUS operation that MySQL does not support natively.

Ids	Employee_Name	Type	Dept_Id
4	Nowrin	O	1

4. RIGHT JOIN (RIGHT OUTER JOIN)

RIGHT JOIN: Returns ALL rows from the RIGHT table, plus matching rows from the LEFT table. Where no match exists in the left table, the left-side columns are filled with NULL.

4.1 RIGHT JOIN vs LEFT JOIN

RIGHT JOIN is the mirror image of LEFT JOIN. Any RIGHT JOIN query can be rewritten as a LEFT JOIN by swapping the table order. In practice, LEFT JOIN is far more common in professional SQL, but RIGHT JOIN is important to understand for reading legacy queries and for working with the Manager table we introduced in the lab.

QUERY 7 — RIGHT JOIN — All Managers, Even Those Not in Employee Table

Goal: Show all managers from the Manager table, with matching Employee data if it exists

```
--sql
- The Manager table has: Ids 1,2,4 (match Employee) + 8,9,10 (no match in Employee)
-- RIGHT JOIN preserves all Manager rows
-- Employees 3,5,6,7 appear in Employee but not Manager -> excluded (they are LEFT side)
SELECT
    E.Name  AS Employee_Name,
    E.Basic_salary AS Employee_Salary,
    M.Name  AS Manager_Name,
    M.Managerial_Allowance,
    M.Dept_Id  AS Manager_Dept
FROM Employee E
RIGHT JOIN Manager M ON E.Ids = M.Ids
ORDER BY M.Dept_Id;
```

Expected : 6 rows (all manager rows)

Output Table:

Employee_Name	Employee_Salary	Manager_Name	Managerial_Allowance	Manager_Dept
Nasim	15000	Nasim	20000	1
		Aslam	50000	1
Arshad	12000	Arshad	25000	2
		Tasmia	70000	2
Nowrin	10000	Nowrin	20000	3
		Samara	60000	3

WHY THIS MATTERS: In any organization, managers are often tracked in a separate table for administrative purposes but not every manager may have a corresponding record in the main Employee table due to data entry gaps, legacy records, or system migration issues. A regular INNER JOIN would silently hide these inconsistencies by returning only the 3 matched rows (Nasim, Arshad, Nowrin), giving the false impression that all managers are properly registered. RIGHT JOIN solves this by preserving every row from the Manager table regardless of whether a match exists in Employee exposing the 3 unmatched managers (Aslam, Tasmia, Samara) with NULL on the left side. This makes RIGHT JOIN an essential tool for data auditing, integrity checks, and identifying orphaned or incomplete records across related tables.

QUERY 8 — RIGHT JOIN — Managers Not in Employee (Anti-Join)

Goal: Identify managers who appear in the Manager table but NOT in the Employee table

```
--sql
-- RIGHT JOIN + WHERE left-side IS NULL = Right Anti-Join
-- Finds Manager rows with no corresponding Employee record
SELECT
   M.Ids  AS Manager_Ids,
   M.Name  AS Manager_Name,
   M.Managerial_Allowance,
   M.Dept_Id
FROM Employee E
RIGHT JOIN Manager M ON E.Ids = M.Ids
WHERE E.Ids IS NULL;    -- Only managers with no Employee record
```

Expected: 3 rows — Aslam (8), Tasmia (9), Samara (10)

These Ids exist in Manager but NOT in Employee — orphaned manager records

Output table:

Manager_Ids	Manager_Name	Managerial_Allowance	Dept_Id
8	Aslam	50000	1
9	Tasmia	70000	2
10	Samara	60000	3

WHY THIS MATTERS: This query acts as a data quality audit tool. In a properly maintained system with a foreign key from Manager.Ids to Employee.Ids, these orphaned records would not exist. RIGHT JOIN + IS NULL exposes such inconsistencies when foreign keys are not enforced.

QUERY 9 — RIGHT JOIN with Aggregation — Department Manager Payroll Summary

Goal: For each department, show total managerial allowance and count of managers.

```
-- RIGHT JOIN ensures all departments appear even if no manager is assigned
SELECT
   D.Name  AS Department,
   COUNT(M.Ids)  AS Manager_Count,
   IFNULL(SUM(M.Managerial_Allowance), 0) AS Total_Managerial_Budget
FROM Department D
RIGHT JOIN Manager M ON D.Dept_Id = M.Dept_Id
GROUP BY D.Dept_Id, D.Name
ORDER BY Total_Managerial_Budget DESC;
```

Output table:

Department	Manager_Count	Total_Managerial_Budget
BBA	2	95000
ETE	2	80000
CSE	2	70000

WHY THIS MATTERS: In payroll and budget planning, it is critical to account for every manager's allowance even those assigned to departments that may have incomplete or missing records in the Department table. A regular INNER JOIN would silently drop any manager whose Dept_Id does not match an existing department, causing the total budget figures to be understated. RIGHT JOIN ensures all rows from the Manager table are preserved, so no managerial allowance is left out of the calculation. Wrapping SUM() inside IFNULL(..., 0) further guarantees that departments with no managers display 0 instead of NULL, producing a clean and complete payroll summary. This patternRIGHT JOIN combined with aggregation is a standard approach in financial reporting,

HR dashboards, and audit trails where missing or unmatched data must be surfaced rather than silently ignored.

5.1 FULL JOIN (FULL OUTER JOIN)

FULL JOIN: Returns ALL rows from BOTH tables. Where no match exists on either side, the non-matching columns are filled with NULL.It is the union of LEFT JOIN and RIGHT JOIN results.

5.1 FULL JOIN in MySQL

MySQL does not support FULL OUTER JOIN syntax directly. The standard workaround is to combine a LEFT JOIN and a RIGHT JOIN using UNION, which automatically deduplicates rows that appear in both result sets.

Full join syntax similar like this:

```
-- Full Join: Cannot be done in MySQL (can be done in Oracle)

SELECT * FROM Employee E
FULL JOIN Manager M
ON E.Ids = M.Ids;

-- Corresponding MySQL Syntax

SELECT * FROM Employee E
LEFT JOIN Manager M
ON E.Ids = M.Ids
UNION
SELECT * FROM Employee E
RIGHT JOIN Manager M
ON E.Ids = M.Ids;
```

QUERY 10 — FULL JOIN — Complete Employee-Manager Universe
Goal: Show ALL employees AND all managers matched where possible, NULLs where not

```
SELECT
   E.Ids AS Ids,
   E.Name  AS Employee_Name,
   E.Basic_salary AS Employee_Salary,
   M.Name AS Manager_Name,
   M.Managerial_Allowance
FROM Employee E
LEFT JOIN Manager M ON E.Ids = M.Ids
```

```
UNION

SELECT
   M.Ids,
   E.Name,
   E.Basic_salary,
   M.Name,
   M.Managerial_Allowance
FROM Employee E
RIGHT JOIN Manager M ON E.Ids = M.Ids

ORDER BY Ids;
```

Expected Output: 10 unique rows

Ids	Situation
1, 2, 4	Exist in both `Employee` and `Manager` tables
3, 5, 6, 7	Exist in `Employee` only — not assigned as managers
8, 9, 10	Exist in `Manager` only — no corresponding employee record

Output table:

Ids	Employee_Name	Employee_Salary	Manager_Name	Managerial_Allowance
1	Nasim	15000	Nasim	20000
2	Arshad	12000	Arshad	25000
3	Fahima	12000		
4	Nowrin	10000	Nowrin	20000
5	Rafiq	45000		
6	Salam	20000		
7	Zia	15000		
8			Aslam	50000
9			Tasmia	70000
10			Samara	60000

WHY THIS MATTERS: In a real organization, employee and manager data are often maintained separately and they do not always stay in sync. A regular `INNER JOIN` would return only the 3

matched rows (Ids 1, 2, 4), hiding the fact that some employees are never assigned managerial roles and some managers have no employee record at all. By combining `LEFT JOIN` and `RIGHT JOIN` with `UNION`, we simulate a FULL OUTER JOIN which MySQL does not natively support and retrieve all 10 unique rows across both tables. This makes the pattern invaluable for data reconciliation, auditing, and detecting missing or orphaned records across related tables.

QUERY 11 — FULL JOIN — Data Reconciliation: Finding Records Exclusive to Each Table

Goal: Find all records that exist in only ONE table (not matched anywhere)

```
SELECT
   E.Ids  AS Ids,
   E.Name  AS Name,
   'Employee Only' AS Status
FROM Employee E
LEFT JOIN Manager M ON E.Ids = M.Ids
WHERE M.Ids IS NULL

UNION

SELECT
   M.Ids,
   M.Name,
   'Manager Only'  AS Status
FROM Employee E
RIGHT JOIN Manager M ON E.Ids = M.Ids
WHERE E.Ids IS NULL

ORDER BY Ids;
```

Output table:

Ids	Name	Status
3	Fahima	Employee Only
5	Rafiq	Employee Only
6	Salam	Employee Only
7	Zia	Employee Only
8	Aslam	Manager Only
9	Tasmia	Manager Only
10	Samara	Manager Only

WHY THIS MATTERS: This query isolates the **symmetric difference** between `Employee` and `Manager` records that exist in exactly one table but not both. Unlike Query 10 which shows the full picture, this query surgically filters out all matched rows using `WHERE` on `NULL` values, exposing only the anomalies. The first half catches employees who have never been assigned a managerial role, while the second half catches managers with no corresponding employee record, a classic sign of data inconsistency. Together, Queries 10 and 11 form a complete **data audit toolkit**: one to visualize the entire universe, and one to isolate the problems.

6. Comparative Summary & Decision Guide

6.1 Which JOIN to Choose?

Join Type	Use When	What You Lose If You Use Wrong Join
INNER JOIN	You only want matched rows	Unmatched rows silently disappear
NATURAL JOIN	Schema is consistent, you want brevity	Breaks silently if column names diverge
LEFT JOIN	All left rows must be preserved	Right-only rows are excluded
RIGHT JOIN	All right rows must be preserved	Left-only rows are excluded
FULL JOIN	Complete picture of both tables needed	Must be emulated in MySQL using UNION

6.2 NULL Behavior in Outer Joins

The appearance of NULL in an outer join result is not an error, it is the join communicating that no matching record exists. This NULL carries semantic meaning:

- NULL in Employee columns after a RIGHT JOIN means: this manager has no Employee record of a data gap.
- NULL in Teacher_Details after a LEFT JOIN means: this employee is not a teacher structurally valid.
- NULL in the Department after a LEFT JOIN means: this department has no employees important for completeness reports.

Always use IFNULL() when NULLs from outer joins would distort aggregations like SUM() or AVG(). NULL + any number = NULL in MySQL arithmetic.

6.3 UNION vs JOIN

Operation	UNION	JOIN
Direction	Vertical: stacks rows from two queries	Horizontal: adds columns from another table
Requirement	Both queries must have same column count	Tables must have relatable columns
Duplicates	UNION removes duplicates; UNION ALL keeps them	Duplicates depend on join condition and data
Used For	Combining same-shaped result sets; FULL JOIN workaround	Combining related tables into enriched rows

Conclusion

SQL JOIN operations are the fundamental mechanism by which normalized relational data is re-assembled for querying. The key insights from this assignment are:

1. NATURAL JOIN eliminates boilerplate in consistent schemas but is fragile and must be used carefully in maintained production environments.
2. LEFT JOIN is the workhorse of reporting; it ensures no record from the primary table is lost, and its anti-join pattern (WHERE right.col IS NULL) is the MySQL equivalent of MINUS.
3. RIGHT JOIN is LEFT JOIN's mirror equally powerful, and together they reveal data asymmetries between tables that INNER JOIN would silently hide.
4. FULL JOIN gives the complete union of both datasets, exposing every matched and unmatched record. In MySQL, it requires a UNION of LEFT and RIGHT JOINs.
5. NULL in outer join results is not an error, it is a meaningful signal that a record exists in one table but not the other, enabling powerful auditing and data reconciliation queries.

Mastering these join types transforms a developer from someone who can query existing data into someone who can audit, validate, and derive insights from the relationships between tables the true power of the relational model.

— End of Assignment —